

About this program:

This is a minimal, RAG demo you can run on your laptop without any paid API. It uses:

1. Sentence Transformers for embeddings (all-MiniLM-L6-v2)
2. FAISS as the vector index
3. A small, local T5 model (t5-small) to generate answers from retrieved context

What this program demonstrates

RAG = Retrieval-Augmented Generation.

This script shows a minimal RAG pipeline:

Index your local .txt documents (turn them into vectors).

Retrieve the most relevant chunks for a question using a vector index (FAISS).

Generate an answer using a small local language model (T5) grounded on the retrieved context.

The data/ folder holds plain-text files you'll search over.

What you'll get

Index a few .txt files from a folder (data/)

Ask a question

It retrieves the most relevant chunks (R) and lets the T5 model generate an answer grounded in those chunks (G)

You'll see:

1. The question
2. The top retrieved chunks (from your data/ files)
3. The generated answer grounded in those chunks

Try changing the question in the __main__ section or pass your own files into data/.

Try your own docs

Drop any .txt files into data/—for example:

Exported notes

README / docs

Policy or design docs

Rerun the script to index and ask questions.

Notes & tips

First run downloads models (small). Subsequent runs are fast.

If you want better outputs, swap `t5-small` → `google/flan-t5-base` (bigger, better).

Optional: Version using OpenAI/other APIs

If you have an API key and want better answers, you can keep the same retrieval pipeline and call an API LLM (e.g., GPT-4o) with a prompt built from the retrieved chunks. Let me know and I'll give you a drop-in replacement for the `LocalGenerator` class.

RAG Demo — Key Components in the Code (and Everything Below)

Key components in the code

This section describes how the minimal Retrieval-Augmented Generation (RAG) sample works: it reads your local `.txt` files, chunks them, builds dense embeddings, indexes them with FAISS for fast similarity search, retrieves the top-k relevant chunks for a question, and then uses a small local language model (T5) to generate an answer grounded in that retrieved context.

1) Reading and chunking documents

- `read_text_files(folder)`: Recursively reads all `.txt` files and returns a list of (filename, full_text).
- `simple_chunk(text, chunk_size, overlap)`: Splits long documents into overlapping word-based chunks so retrieval can find precise segments rather than entire documents.
- `build_corpus(docs, ...)`: Transforms each file into multiple chunks with lightweight metadata — (source_filename, chunk_text, chunk_index_within_file).

2) Embeddings + FAISS index

- `VectorIndex`: wraps a `SentenceTransformer` embedder (default: "sentence-transformers/all-MiniLM-L6-v2").
- `fit(corpus)`: Encodes every chunk into a normalized embedding vector and builds a FAISS `IndexFlatIP` (inner product used as cosine similarity because vectors are normalized).

- `query(question, top_k)`: Encodes the question, searches the index, and returns the indices + similarity scores for the most relevant chunks.

3) Local generator (T5)

- `LocalGenerator`: loads a small sequence-to-sequence model (default: t5-small) and its tokenizer.
- `generate(prompt)`: Produces an answer string deterministically (greedy) with a cap on `max_new_tokens` to keep responses short and fast.

4) Prompt construction

- `build_prompt(question, context_chunks)`: Builds an instruction-like prompt that lists [Context 1..N] followed by the question.
- The instruction asks the model to answer using ONLY the provided context, and to say 'I don't know' if the answer is not present.
- Prefixing with a simple task cue (e.g., "summarize:") nudges T5 to follow instructions better.

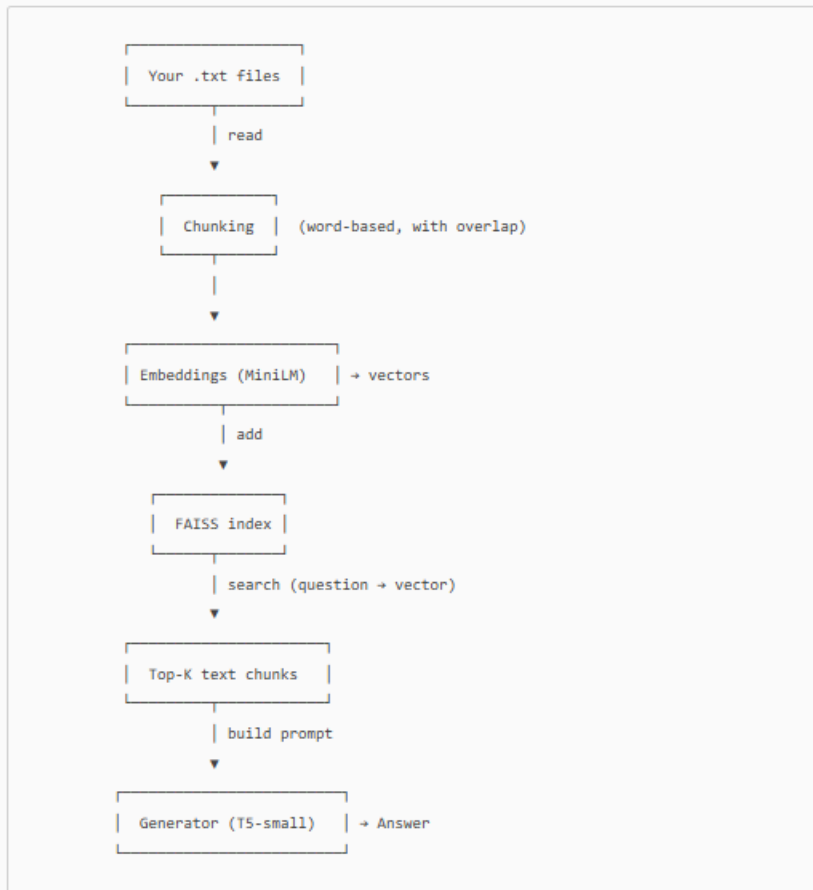
5) RAG pipeline orchestration (`run_rag`)

1. Read and chunk documents from the data/ folder.
2. Encode chunks, build FAISS index.
3. Retrieve top-k chunks for the question.
4. Construct a compact prompt from the retrieved chunks + question.
5. Generate the final answer using T5 and print the question, retrieved chunks, and answer.

End-to-end flow (conceptual)

Your .txt files → 2. Chunking → 3. Embeddings (MiniLM) → 4. FAISS index → 5. Retrieve top-K chunks → 6. Build prompt → 7. T5 generate → Answer

End-to-end flow (visual)



What the script prints (and why)

- **QUESTION:** The query posed to the pipeline, to keep execution transparent.
- **TOP CONTEXT CHUNKS:** The specific retrieved chunks with their similarity scores, so you can audit retrieval quality.
- **ANSWER:** The model output generated from the prompt that contains only those retrieved chunks.

Useful levers you can tweak

- **Question:** Change the question argument in `run_rag()`.
- **Retrieval size:** Adjust `top_k` (e.g., 3–8).
- **Chunking:** Tune `chunk_size` and `overlap` for your documents; larger chunks give more context per hit, smaller chunks allow finer retrieval.
- **Generator:** Swap `t5-small` → `google/flan-t5-base` for better answers (needs more memory).
- **Hardware:** If you have a GPU, `load/model.to("cuda")` and push inputs to CUDA for speedups.

Common gotchas

- First run downloads models; subsequent runs are offline and faster.
- If answers look off-topic, inspect the printed retrieved chunks — adjust `chunk_size`, `overlap`, or `top_k`.
- T5-small is intentionally tiny; for higher fidelity use a slightly larger instruction-tuned model (e.g., FLAN-T5).

How this maps to real-world DevOps usage

- Index runbooks, infra-as-code READMEs, incident docs; retrieve to answer on-call “how-to” questions with sources.
- Add a CLI wrapper so engineers can query: `rag query "How do I rotate SSH keys?"`.
- Automate postmortems by retrieving context from logs or prior incident write-ups and summarizing key points.

Quick experiment to validate retrieval

6. Create `data/python-venv.txt` with Windows/macOS/Linux `venv` steps.
7. Ask: "How do I create and activate a Python virtual environment on Windows?"
8. Re-run: You should see that the new `venv` chunk is retrieved and the answer becomes more specific.